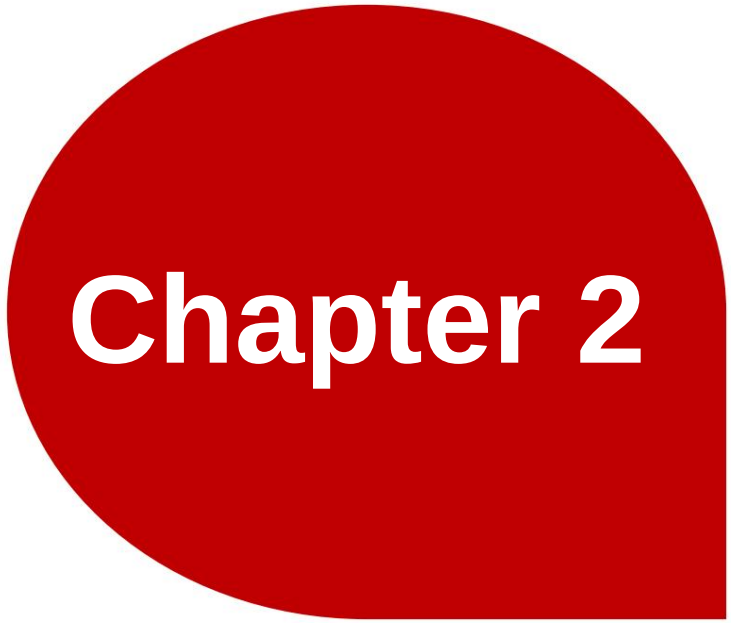
POO Java

Cours : Programmation Orientée Objets - L2-S2 Informatique, UMBB

Gaceb
miec.infodz@gmail.com



History of Java
IDE
Entrances/Exits
The variables
Arrays (vectors and matrices)
Methods
References and parameter passing
Execution time

Introduction to the Java language

A brief history of Java

Origin

Created by Sun Microsystems

Target: embedded systems (vehicles, household appliances, etc.) using dedicated languages that are incompatible with each other

Key dates

1991: Introduction of the "Oak" language by James Gosling 1993: Rise of the Web thanks to Mosaic (the idea of adapting Java to the Web takes hold)

1995: JDK Alpha and beta, Development of the HotJava software in Java allowing the execution of applets 1996: JDK 1.0,

NetscapeTM Navigator 2 incorporates a Java 1.0 virtual machine 1997: JDK 1.1, A first step towards an industrial version, Web-oriented 1998: J2SE 1.2, becomes general-purpose, renamed Java 2, 3 times more packages than Java 1.0.

1999: Industrial version of Java

2000: J2SE 1.3, 2002: J2SE 1.4, 2004: J2SE 5.0, 2006: Java SE 6, 2011: Java SE 7 2014: Java SE 8, 2017: Java SE 9, 2018: Java SE 10 and 11, 2019: Java SE 12 and 13 2020: Java SE 14 and 15, 2021: Java SE 16 and 17

Compilation and execution

The source code cannot be executed directly.

In Java, the source code is translated into a Java-specific language: "bytecode." This is the language of the Java Virtual Machine (JVM).

- This language is machine independent, therefore highly portable, since there are JVMs for all architectures (operating systems).
- Write one and run anywhere. Be careful with the CLASSPATH environment variable.

Our source program is called **HelloWorld.java.**

After compilation, we get a new **HelloWorld.class file,** whose content is bytecode.

- Command line: **javac HelloWorld.java**

The bytecode must be executed by a JVM (Java Virtual Machine).

- This virtual machine is simulated by an interpreter, which reads each instruction of the program (bytecode) and
- translates it into the language of the computer's processor and launches its execution.
- Command line: **java HelloWorld**

Main stages of development

1. Creation of source code

In a MyClass.java file, MyClass is the name I gave to my main class Tools: text editor, IDE **2.**

Compilation

to Byte-Code Creating a

MyClass.class file from source code

Tool: Javac compiler

3. Diffusion on the target architecture

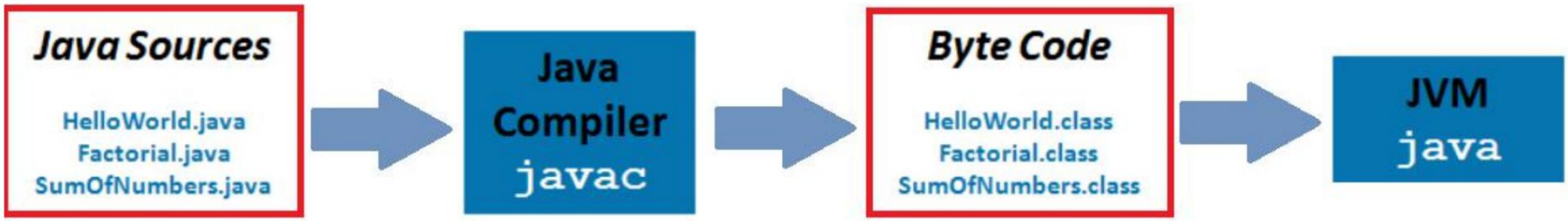
Bytecode transfer only Tool:

network, disk, etc.

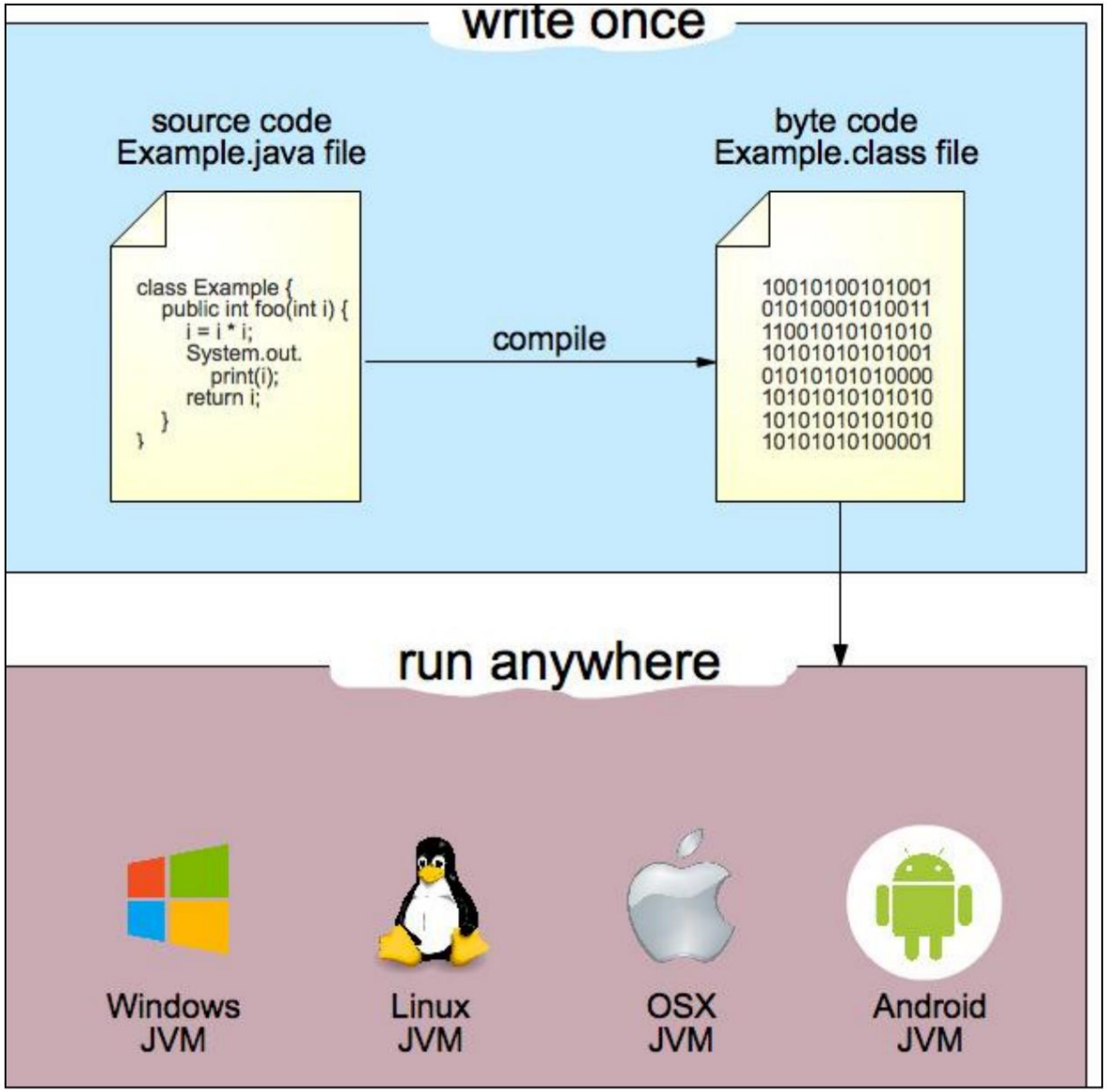
4. Execution on the target machine

Byte-Code Execution

Tool: Java Virtual Machine (JVM) = same code works on different operating systems (Windows, Mac, Linux, Android)



Main stages of development



Development tools

Most Used Java IDEs

1. Eclipse (will be used in your practical sessions)
2. IntelliJ IDEA 3. Netbeans 4. BlueJ 5. JGRASP 6. Android Studio 7. Java Inventor 8. Codenvy
9. JBuilder
10. Jcreator
11. DrJava
12. Jdeveloper
13. Others

An IDE *(Integrated* Development Environment) is an environment that allows you to create and run programs using the tools provided by that environment. We are therefore talking about tools integrated into the environment.

An IDE to simplify application development will therefore provide a set of tools.

Bibliography

1. Thinking in Java 2nd French edition (608 pages): http://bruce-eckel.developpez.com/livres/java/traduction/tij2/tij2.pdf

2. Thinking in Java 3rd French edition: http://bruce-eckel.developpez.com/livres/java/traduction/tij3/tij3fr.zip

3. The Java site: https://www.oracle.com/technetwork/java/javase/documentation/index.html

4. API (reference): see other more recent versions https://docs.oracle.com/javase/7/docs/api// 5. Sun

Tutorial: https://docs.oracle.com/javase/tutorial/

6. Courses and examples: https://java.developpez.com/

7. Learn Java and C++ in parallel, Jean-Bernard Boichat Publisher: Eyrolles Edition: 2003 - 742 pages - ISBN: 2212113277

Java language characteristics (+ and -)

Java is an object-oriented language with

classes (objects are described/grouped in classes). Its syntax is very similar to

C and C++. It comes with the SDK (Software

Development Kit), development tools, and packages. It is portable, as its slogan indicates: "write once, run

everywhere." Advantages of the JVM: Portability: JVMs exist for all operating systems on

the market.

The exported bytecode is lightweight.

Security, the JVM runs many checks on the bytecode.

SUN defines Java as: simple, safe, OO, portable, distributed, efficient, interpreted, multi-tasking, robust, dynamic.

Disadvantages of JVM:

Not as fast as a native program: Slow execution (compilation + interpretation) but other alternatives exist.

Gourmet in memory

No operator overloading like in C++

Some rules and elements of language

A class name always begins with a capital letter.

Words within an identifier begin with a capital letter: HelloWorld.

Constants are in uppercase.

Properties and methods begin with a lowercase letter.

Add comments, the syntax is the same as C (/* ... */ or //).

Elements of language

Blocks: Opening and closing with **"{"** and "}".

Arithmetic: **+,-,*,/,%** (modulo)

Logics: **&&** (logical and), **||** (logical or), !(inverse).

Comparisons: **==** (equality), **!=** (difference), **>, <, <=, >=**

Unary operators: **++, --**

Assignments: **+=, -=, *=, /=**

a = a + 1; equivalent to a++; equivalent to a+=1

First example program in Java

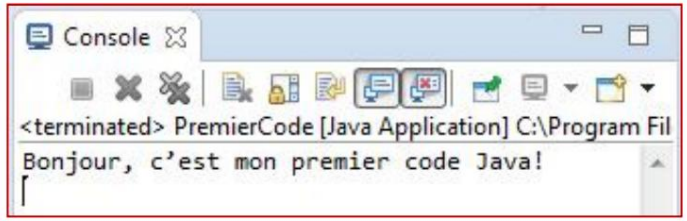
Write a program that displays the following message on the screen

"Hello, this is my first Java code!"

Create a PremierCode.java file Enter

the following code to compile

```
public class PremierCode {
    public static void main(String[] args) {

        System.out.println("Bonjour, c'est mon premier code Java!");
    }
}
```



public class PremierCode: Name of the class public

static void main: The main function equivalent to the main function in C/C++

String[] argv: Allows you to retrieve arguments passed to the program when it is launched

System.out.println("Hello… ") : Display method in the console window Equivalent to printf in C and cout<< in C++ Allows you to display the message between () then a line break To display a message without a line break use: System.out.print("msg ")

Line break can be done by inserting at the end of msg \n: System.out.print("msg \n")

Learn to read keyboard input

Scanner class usage (import java.util.Scanner;)

Example: reading values entered by the user

```
import java.util.Scanner; public class
Main {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter an integer: ");
        int i = sc.nextInt();
        System.out.println("Enter a string: "); //Empty the line before reading another
        one sc.nextLine(); String str = sc.nextLine(); System.out.println("END!
        ");


    }}
```

One point to note, however, is that the nextLine() method retrieves the contents of the entire line entered and replaces the "read head" at the beginning of another line. However, if you invoke a method like nextInt(), nextDouble() and you invoke it directly after the nextLine() method, it will not prompt you to enter a string: it will empty the line started by the other instructions.

Primitive types

Primitive types: are non-object data types found in many other programming languages.

Integers: byte (1 byte) - short (2 bytes) - int (4 bytes) – long (8 bytes)

Floating points (IEEE-754 standard): float (4 bytes) - double (8 bytes)

Booleans: boolean (true or false)

Characters: char (16-bit Unicode encoding)

Variable declaration: <type> <identifier>; int a;

float b; double c, d, e;

Primitive type wrappers: Each of these types corresponds to an object type with conversion methods, also called a “wrap type”.

A wrapper class allows you to encapsulate a value of a primitive type within an object. The value is stored as an attribute. Each class allows you to manipulate the value through methods (conversions, etc.).

Character for char, Byte for byte, Short for short, Long for long Integer for int

Float for float

Double for

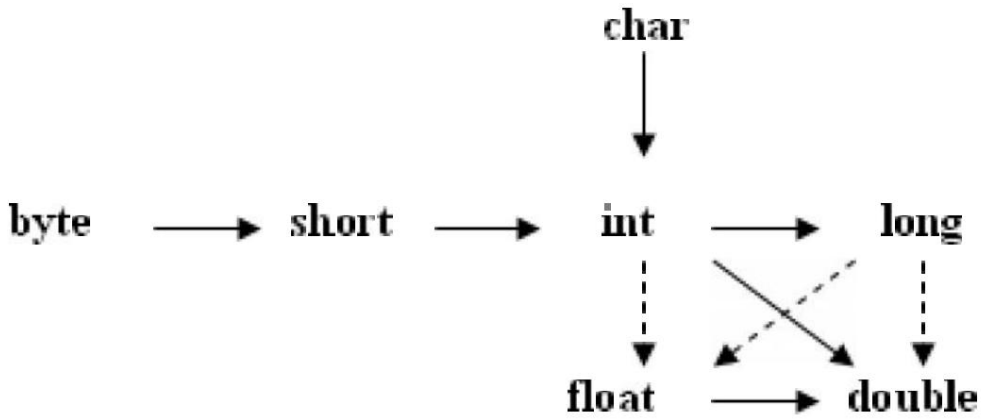
double Boolean for

boolean

Primitive type conversions (casting)

Conversions to a type with a larger or equal representation size are done automatically (implicit conversion) without needing to be specified.

In the diagram below, the solid arrows indicate automatic conversions without loss of information, the dotted arrows indicate automatic conversions with possible loss of information:



Other conversions may result in loss of information, and must be specified by a *cast operator (explicit casting).*

Example :

```
1. double x = 45675.7;
2. byte y = (byte)x; // y is 107
3. int z=(int) x;                    // z is 45675
```

Non-primitive types: by reference

Other data types are reference types to objects or arrays.

are variables intended to contain addresses of objects or arrays, **In Java: non-primitive variables are manipulated "by reference" while primitive variables are manipulated by value.**

The default value of any attribute of a "by reference" type is null.

Non-primitive types

Tables : **int[] tab1 = new int[7];**

int[] tab2 = {15,10,5,8};

Character strings: **String firstname = "Mohamed";**

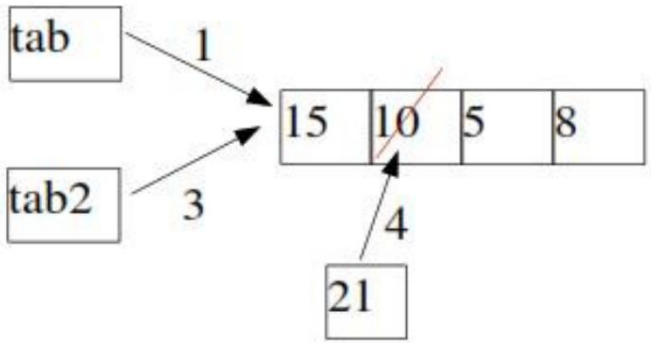
Objects: **class Person { String name; int age; }**

Person P1 = new Person();

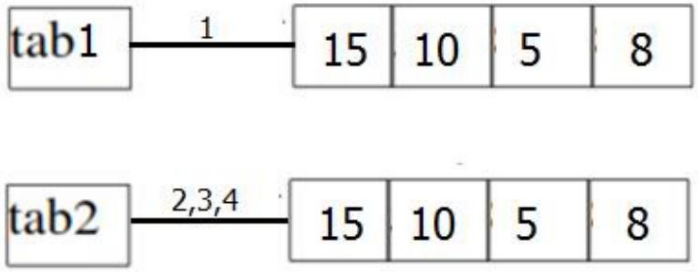
Non-primitive types: by reference

Example: Manipulating a type by reference

```
1:int[] tab = {15,10,5,8};
    T
2:int[] tab2 = new int[4];
3:tab2 = tab;
4:tab[1] = 21;
if(tab==tab2) // vrai
```




```
1:int[] tab = {15,10,5,8};
2:int[] tab2 = new int[4];
3:for(int cpt=0; cpt<4;cpt++)
4:   tab2[cpt] = tab[cpt];
5:if(tab==tab2)
// faux, car 2 références distinctes.
```



Initiation and constant

Initialization

A variable can be assigned a value at the time of its declaration, **int a = 30; boolean b = true;**

This instruction plays the same role as: **int a; a = 30; boolean b; b = true;**

Constants

These are variables whose value can only be assigned once.

They can no longer be modified

They are defined with the final keyword and the variable identifier in MAGISCULE (by convention)

Final

```
example float PI_APP = 3.14F; final int VAR;

VAR= 8; // correct
VAR = 10; // error: VAR is declared final
```

Control structure

Conditional statement (choice)

Treating a case:

```
if (condition) {bloc_inst1;}
```

Example:

```
int a=-1; if(a<0) System.out.println("a is negative!");
```

Treating a case and reverse case:

```
if (condition) {bloc_inst1;} else {bloc_inst2;}
```

Treating two cases and inverse case etc. (The conditions must be complementary):

```
if (condition1) {bloc_inst1;} else if (condition2) {bloc_inst2;}
else {bloc_inst3;}
```

if condition1 is true then inst_block1 is executed, else if condition2 is true then inst_block2 is executed, else then inst_block3 is executed, We can add as many else if statements as needed. Conditional statements can be nested within each other.

Control structures

Limited selection

switch val { case val0 : … val2:… default:…) // default is optional when val != vali

Example

```
int day = 4; switch
(day) {
  case 6: System.out.println("Today is a Friday"); break; case 7: System.out.println("Today is a
  Saturday"); break; default: System.out.println("Looking forward to the weekend"); break; }

// Display: Waiting for the weekend
```

Iterations: three types of loops for, while, do...while

for (initialization; condition; modification){…}

for (Type var: Collection) {…} // New in Java 5

Number of iterations is unknown in advance

while (condition) { … } **do** { … } **while** // test then execute {…}

(condition) // execute {…} then test

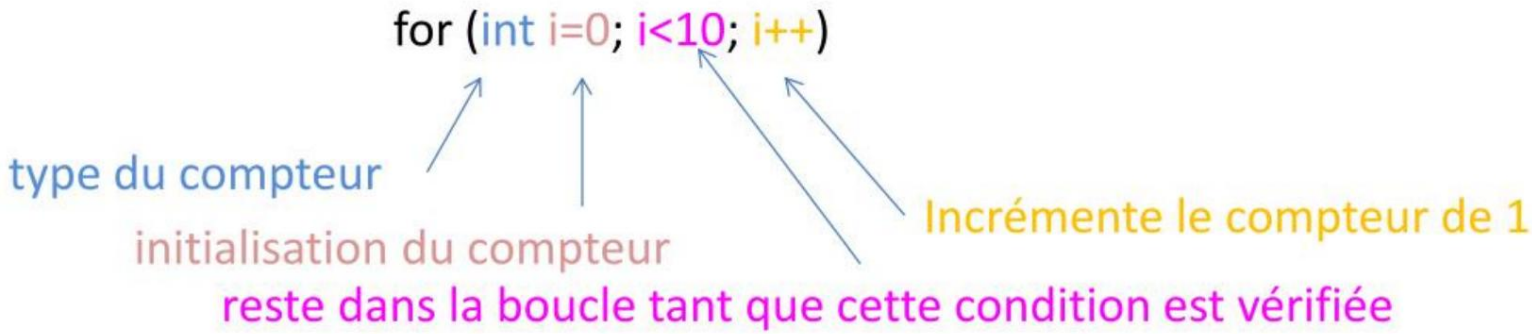
The instructions can be used in the loop:

break; // Force all iterations to stop and exit the loop

continue; // Force the current iteration to stop and move on to the next iteration

Control structure: for loops

Goal : **to perform a number of iterations known in advance**



Example: **sum the first 10 integers**

```
int j = 0;
for (int i=0; i<10; i++)
  j += i;
```

Control structures: while and do…while

Example while(cond){…}

When the 'value' is still less than 10, the loop still runs.

```
int value = 3;
while (value < 10) {
                System.out.println("Value =                    " + value);
                  // Increase the "value" by adding 2
                    value = value + 2;
    } // displays: Value = 3 Value = 5 Value = 7 Value = 9
```

Example do{…} while(cond);

```
  do {
                System.out.println("Value =                    " + value);
                  // Increase the "value" by adding 2
                    value = value + 2;
   } while (value < 2);
// displays: Value = 3
```

Tables (1D, 2D, …)

Arrays

In Java, an array is a data structure containing a set

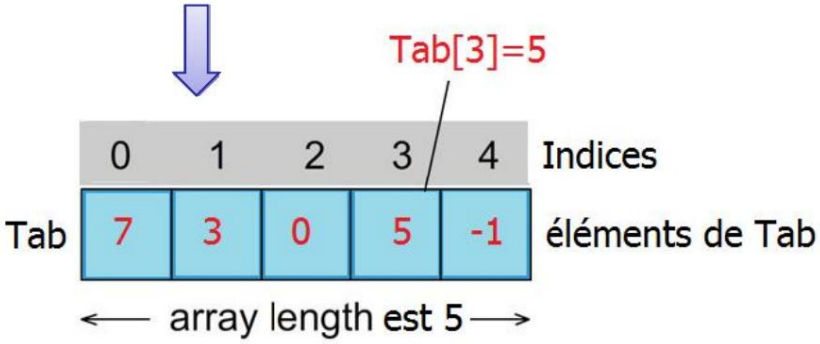
of elements of the same type, stored consecutively in memory.

Its size is fixed when it is created and cannot be changed.

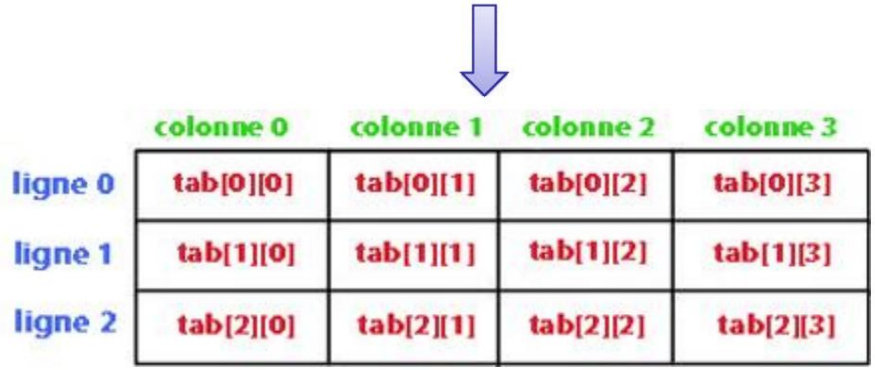
Arrays are considered objects; they can contain:

- Primitive type variables (int, float, double, boolean, char, …)
- References on objects

One-dimensional array



Two-dimensional array



Tables (1D)

Creating a 1D Table: 3 Types of Declaration and Dimensioning

// First type of declaration without giving the number of elements.

```
int[] tab1;
```

// Initialize the array of 3 elements without assigning them a specific value.

```
tab1 = new int[3];
```

// Declare an array, specify the number of elements without assigning a value to them

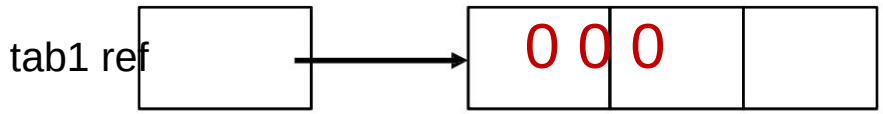
```
double[] tab2 = new double[8];
```

// Declare an array of 4 elements with initialization of their value

```
float[] tab3= {9.6F, 5.7F, 4.5F, 1.2F};
```

Noticed :

Creating an array with new Allocates memory based on the array type and size Initializes the array contents to 0 for simple types tab1.length returns the array size



Tables (1D)

Example of filling and displaying the elements of an array

```
int[] tab = new int[5];                    // creation in memory of an array of 5 elements
// Assigning values to array elements
        tab[0] = 7; tab[1] = 5; tab[2] = 8; tab[3] = 9; tab[4] = 2;
System.out.println("Tab_size=" + tab.length); // Displays the tab size
// Use a for loop to display the items in the tab
//Method 1
        for (int i= 0; i < tab.length; i++) {
                System.out.println("T[" + i+ "] =                    " + tab[i]);
        }
//Method 2: for each (from java 5)
        int k=0;
        for (int t : tab) {
                System.out.println("T[" + k++ + "] =                    " + t);
        }
```

– Example 2: use of for each

```
// Array of 3 String (primitive name type).
    String[] fruits = new String[] { “Banana”, “Apple”, “Orange” };
  // Display the array elements.
      for (String fruit: fruits) { System.out.println(fruit); }
```

Arrays (1D): Java ArrayList

In Java, we need to declare the size of an array before we can use it.

Once an array's size is declared, it is difficult to change it.

To solve this problem, we can use the ArrayList class.

It allows us to create resizable tables.

Before using ArrayLists, we first need to import the java.util.ArrayList package.

Here's how to create array lists in Java:

```
ArrayList<Type> tab = new ArrayList<>();
```

- **For example :**

```
// Create an arraylist of type Integer
ArrayList<Integer> tab = new ArrayList<>();

// Create an arraylist of type String
ArrayList<String> tab = new ArrayList<>();
```

Arrays (1D): Java ArrayList -> Example

```
import java.util.ArrayList; class Main { public
static void main(String[] args){ // create an empty ArrayList of type String

ArrayList<String> languages = new ArrayList<>();
// Add an element to the languages array (
    languages.add("Java"); languages.add("Python"); languages.add("C++");

// Retrieve the second element of the languages array
     String str = languages.get(1); System.out.print("Element
     at index 1: "                                          + str);
// Modify the 3rd element
     languages.set(2, "JavaScript"); //Display the elements of
the languages table
System.out.println("ArrayList: "                          + languages); } }
```

Output display

```
Element at index 1: Python
ArrayList: [Java, Python, JavaScript]
```

Tables (1D): Vector

The Vector class is an implementation of the List interface that allows us to create resizable arrays similar to the class

ArrayList: Vector<Type> vect = new Vector<>();

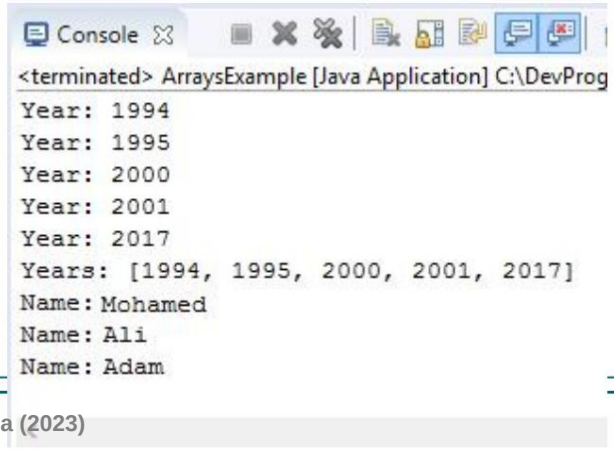
– **For example:**

```
import java.util.Vector; class Main {

  public static void main(String[] args) {

Vector<String> tab= new Vector<>(); tab.add("Dog");

tab.add("Chat");
   // Using an index
tab.add(2, "rabbit");
System.out.println("Vector: "                    + tab);
} }
```

Tables (1D): Lists

Some utility methods of Arrays and List class. import java.util.Arrays; // return to package further import java.util.List;

```
public class ArraysExample {
    public static void main(String[] args) {
        int[] years = new int[] { 2001, 1994, 1995, 2000, 2017 }; // Sort Arrays.sort(years); for
        (int year:
        years)
        { System.out.println("Year: " + year); }
        // Convert an array to a string
        String yearsString = Arrays.toString(years);
        System.out.println("Years: " + yearsString);
        // Create a list (List) of multiple values.
        List<String> names = Arrays.asList("Mohamed", "Ali", "Adam"); for (String name: names)
        { System.out.println("Name: " + name); }
    }
}
```



Multidimensional arrays

2D rectangular array (matrices)

The matrix is composed of several one-dimensional arrays of the same size

Its declaration is made with two bracket operators [ ] [ ] .

The first bracket is used to indicate the number of lines (number of tables at a dimension), the second bracket is used to indicate the number of columns (row size)

// 1) Declare a matrix without giving its size.

Type [][] mat1;

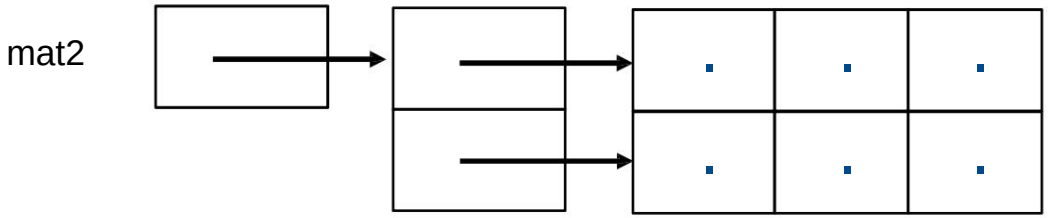
mat1 null

// Initialize the size to 2 rows and 3 columns without assigning them a specific value.

mat1 = new Type[2][3];

// 2) Declare a matrix, specify the size without assigning a value to them

Type[][] mat = new Type[2][3];



Multidimensional arrays

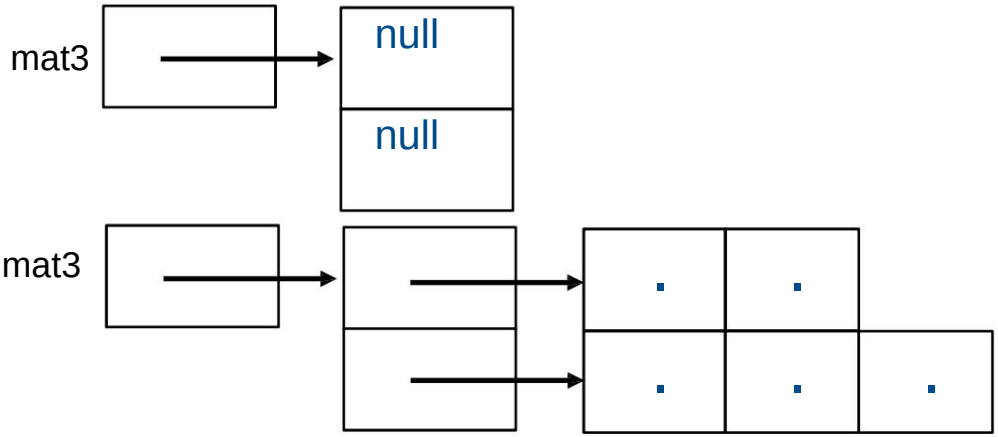
Non-rectangular 2D array (matrices)

There is no requirement that the referenced arrays have the same dimension; this type of array is called jagged arrays or jagged arrays in Java:

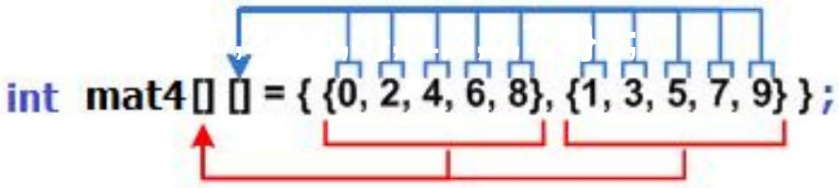
Type[][] mat3 = new Type[2][];

mat3[0]=new **Type** [2];

mat3[1]=new **Type** [3];



Creating and Initializing Values of a 2x5 Matrix



Nous changeons de colonne par le biais de la première paire de crochets

Nous choisissons le terme d'un tableau grâce à la deuxième paire de crochets

Multidimensional arrays

Loading values into mat

```
for (int i = 0;i < mat.length; i++) { for (int j = 0;j <
        mat[i].length;j++) { mat[i][j] = i + j; //For example


        }}
```

Displaying matte elements

```
for (int i = 0;i < mat.length; i++) { for (int j = 0;j <
        mat[i].length;j++) {
            System.out.println("mat["+i+"]["+j+"]="+ mat[i][j]); } }
```

Methods (functions)

A method:

It is used to perform certain actions, it allows you to define a code that is repeated only once and use it several times.

It must be declared in a class. In Java, there are two types of methods (functions): instance methods and class methods (must start with the keyword static). The second type of methods can be called (executed) without the need to instantiate the class.

Declaration of a method (Java syntax):

– <qualifiers><result type><function name> (<formal parameter list>){< **instruction block>}**

The function header (signature) is marked in blue, its definition (body) is in red, the return type can be primitive or object. The list of formal parameters (arguments) can contain: constants, variables (primitive or object type), arrays, expressions, other methods returning a value. The list can also be empty.

Qualifiers are keywords that allow you to modify the ***visibility*** (public, private, etc.) or the ***operation (static, etc.)*** of a method, such as static

Method call: Method calls in Java are carried out very classically with **effective (real) parameters,** the number of which must be the same as that of the formal parameters, and the type must be either the same or a compatible type that does not require casting.

Methods (functions)

Example of creating two methods display and sum

```
public class MyClass {
    void display(String message) { // method returns nothing in output type channel
        System.out.println("Welcome to WayToLearnX!"); }
    int sum (int a, int b) { return a+b;} //returns an integer value public static void main(String[] args) {

        display("Hello everyone!"); // call the display method // call the sum method
        int s1=sum(5,4);
        int s2=sum(8,9);
    }}
```

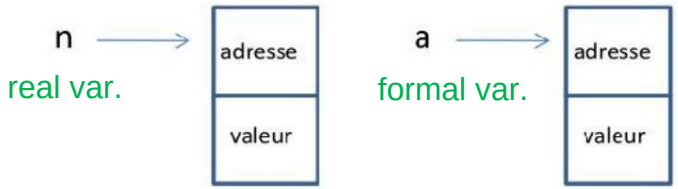
Two modes of transmission or passing of parameters in Java

By value: concerns the parameters of primitive types

the values of n and a are duplicated

reserved for all elementary types

(int, byte, short, long, boolean, double, float, char).

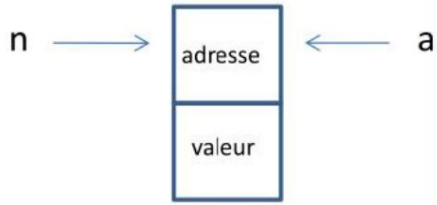


If the value of a changes, the value of n is not affected.

By reference: Object type parameters are passed by reference.

Including arrays: eg. **void display(int[] array){…}**

Real and formal variables have the same reference



It is possible to mix both modes at the same time

Methods (functions)

Method overloading

Allows calling multiple methods with the same name, provided their arguments differ (in type and/or number).

Example int

```
sum( int p1, int p2) { return (p1 + p2); }
Float sum( float p1, float p2){return (p1 + p2);} float sum( float p1, float p2, float p3){return (p1 + p2 +
p3);}
int          sum( float p1, int p2){return (int(p1) + p2);}
```

When called, the appropriate method will be chosen based on the type and number of parameters passed. int

s1=sum(2,3); // this is the first sum that will be called float s2=sum(2.3F, 3.9F, 4.1F); // this is the third sum that will be called

Run Time and Pause

Execution time in sec

```
longStartTime = System.currentTimeMillis();
Block or function();
long endtime = System.currentTimeMillis();
double time = (EndTime - StartTime) / 1000F;
System.out.println("\n Execution time: "  + time+  " dry");
```

Execution time in nano

```
longStartTime = System.nanoTime();
Block or function();
longendtime = System.nanoTime();
double time = (timeEnd - timeStart) / 1000000F;
System.out.println("\n Execution time: "  + time+  "nano");
```

Temporary break of n sec

```
Thread.sleep(n*1000);
```